

Parallel Acceleration of Model Checking for Railway Interlocking Systems

Jacopo Bruscatelli

`jacopo.bruscatelli@stud.unifi.it`

Department of Information Engineering (DINFO)

Università degli Studi di Firenze

Abstract

Model checking in Computer-Based Interlocking (CBI) systems suffers from the combinatorial state space explosion problem, rendering sequential exhaustive safety verification computationally intractable. In this paper, we propose and evaluate parallel implementations of a railway interlocking safety verification kernel using Python Multiprocessing with POSIX Shared Memory and an NVIDIA CUDA kernel using Numba. By compressing system states into primitive 32-bit unsigned integers, state properties are evaluated through single-cycle bitwise operations. Experimental results demonstrate that multicore CPU multiprocessing achieves near-linear scaling ($S_2 = 1.93\times$, efficiency 96.6%) up to the physical core limit, while our massively parallel GPU implementation on an NVIDIA Tesla T4 achieves an execution time of 1.49 ms with a speedup of $\sim 253,000\times$ compared to the sequential baseline.

1 Future Distribution Permission

The author of this report gives permission for this document to be distributed to Unifi-affiliated students taking future courses.

2 Introduction

Modern railway transportation relies on Computer-Based Interlocking (CBI) systems to control switch gears, signals, and track blocks, guaranteeing that trains are routed safely through stations and junctions without conflict. Because these systems are safety-critical, formal verification via model checking is mandated to prove that the system can never reach an unsafe configuration, such as a collision or derailment.

The main challenge in railway model checking is the **state space explosion problem**. A station layout with T tracks, S switches, and R potential routes generates a reachability graph whose total number of reachable states grows combinatorially. Exhaustive verification of safety invariants across millions of states sequentially requires substantial computation time, often exceeding feasible development cycles.

2.1 Railway Model Checking Problem Formulation

An interlocking topology consists of a set of switches $W = \{w_1, \dots, w_m\}$ and a set of routes $\mathcal{R} = \{r_1, \dots, r_n\}$. Each route r_i requires a specific switch alignment configuration and locks associated track sections. Two routes r_i and r_j are defined as *rivals* if their trajectories share one or more track blocks or switches:

$$(r_i, r_j) \in \text{Rivals} \iff \text{Blocks}(r_i) \cap \text{Blocks}(r_j) \neq \emptyset \quad (1)$$

For each reachable system state S_k , the safety verification kernel evaluates two fundamental invariants:

1. **Route Mutual Exclusion Invariant:** No two conflicting routes can be simultaneously in a reserved or occupied state (≥ 2):

$$\forall(r_i, r_j) \in \text{Rivals}, \quad \neg(\text{State}(r_i) \geq 2 \wedge \text{State}(r_j) \geq 2) \quad (2)$$

2. **Switch Locking Consistency:** When an itinerary is locked ($\text{State}(r_i) = 2$), all underlying switch points must strictly match the required routing alignments.

2.2 Bitwise State Packing Layout

Traditional object-oriented graph representations cause substantial memory footprint expansion and pointer-indirection penalties. In this work, the complete interlocking state is compacted into a primitive 32-bit unsigned integer (`uint32`):

- **Turnout Positions (Bits $0 \dots N_{sw} - 1$):** 1 bit per switch (0 = Normal, 1 = Reverse).
- **Route Lifecycles (Bits $N_{sw} \dots N_{sw} + 2N_{rt} - 1$):** 2 bits per route, representing the lifecycle states:

$$00_2 = \text{IDLE}, \quad 01_2 = \text{REQUESTED}, \quad 10_2 = \text{RESERVED}, \quad 11_2 = \text{OCCUPIED}$$

This ultra-compact representation ensures entire state arrays fit into cache lines, minimizes RAM-to-CPU bandwidth consumption, and enables bitwise operations ($\&$, $|$, $\wedge$, $\gg$) on hardware functional units.

3 Proposed Approach

We propose and evaluate three architectural variants of the model checking verification kernel:

1. A single-threaded **CPU Sequential baseline** in Python/NumPy.
2. A **CPU Multiprocessing** implementation using POSIX Shared Memory (`multiprocessing.shared_memory`).
3. A massively parallel **NVIDIA CUDA** kernel compiled via Numba.

3.1 Sequential Implementation

The sequential baseline iterates over all generated system states and evaluates both rival-route conflict pairs and switch locking requirements.

Listing 1: Alg 1. Sequential Verification Kernel

```
def compute_sequential_kernel(states, pairs, num_sw, num_rt, sw_mask,
    out_results):
    for idx in range(len(states)):
        state = states[idx]
        switches = state & sw_mask
        routes = state >> num_sw
        is_unsafe = 0

        # Invariant 1: Check conflicting pairs
        for shift1, shift2 in pairs:
            r1 = (routes >> shift1) & 0b11
            r2 = (routes >> shift2) & 0b11
            if r1 >= 2 and r2 >= 2:
                is_unsafe = 1
                break
```

```

# Invariant 2: Check switch alignment consistency
if is_unsafe == 0:
    for r_idx in range(num_rt):
        r_st = (routes >> (r_idx * 2)) & 0b11
        if r_st == 2:
            if (switches ^ (r_idx * 31)) & 0b10101010 == 0:
                is_unsafe = 1
                break
    out_results[idx] = is_unsafe

```

3.2 CPU Multiprocessing with POSIX Shared Memory

In Python, standard threading is constrained by the Global Interpreter Lock (GIL). Multiprocessing overcomes this by spawning isolated processes, but standard inter-process communication (IPC) via pipes or serialization (pickling) introduces severe data-transfer overhead on multi-million element state arrays.

To achieve zero-copy parallelism, we allocate read buffers for **states** and write buffers for **results** within POSIX shared memory segments. Worker processes attach to these named buffers, construct NumPy array views, and process disjoint index ranges concurrently without lock contention.

Listing 2: Alg 2. Shared Memory Multiprocessing Worker

```

def _shm_worker_range(range_tuple):
    start, end = range_tuple
    shm_s = SharedMemory(name=_SHM_S_NAME)
    shm_r = SharedMemory(name=_SHM_R_NAME)
    states = np.ndarray((_G_LEN,), dtype=np.uint32, buffer=shm_s.buf)
    results = np.ndarray((_G_LEN,), dtype=np.uint8, buffer=shm_r.buf)

    for idx in range(start, end):
        state = states[idx]
        switches = state & _G_MASK
        routes = state >> _G_NSW
        is_unsafe = 0

        for shift1, shift2 in _G_PAIRS:
            if ((routes >> shift1) & 3) >= 2 and ((routes >> shift2) & 3) >= 2:
                is_unsafe = 1
                break

        if is_unsafe == 0:
            for r_idx in range(_G_NRT):
                if ((routes >> (r_idx * 2)) & 3) == 2:
                    if (switches ^ (r_idx * 31)) & 170 == 0:
                        is_unsafe = 1
                        break
            results[idx] = is_unsafe

    shm_s.close()
    shm_r.close()

```

3.3 NVIDIA CUDA Implementation

The model-checking invariant evaluation is embarrassingly parallel across states. Leveraging NVIDIA’s Single Instruction, Multiple Threads (SIMT) architecture, we map each system state to a unique GPU thread in a 1D grid layout ($idx = blockIdx.x \times blockDim.x + threadIdx.x$). The

state vector and topological rival pairs are transferred directly to device memory, eliminating host-device synchronization during execution.

Listing 3: Alg 3. CUDA Safety Verification Kernel (Numba)

```
@cuda.jit
def cuda_safety_kernel(states_arr, pairs_arr, results_arr, mask_val, n_sw
, n_rt):
    idx = cuda.grid(1)
    if idx < states_arr.size:
        state = states_arr[idx]
        switches = state & mask_val
        routes = state >> n_sw
        is_unsafe = 0

        # Route collision check
        num_pairs = pairs_arr.shape[0]
        for p in range(num_pairs):
            shift1 = pairs_arr[p, 0]
            shift2 = pairs_arr[p, 1]
            if ((routes >> shift1) & 3) >= 2 and ((routes >> shift2) & 3)
                >= 2:
                is_unsafe = 1
                break

        # Switch consistency check
        if is_unsafe == 0:
            for r_idx in range(n_rt):
                if ((routes >> (r_idx * 2)) & 3) == 2:
                    if (switches ^ (r_idx * 31)) & 170 == 0:
                        is_unsafe = 1
                        break
            results_arr[idx] = is_unsafe
```

4 Experimental Results

The primary metric used to evaluate performance across implementations is the Speedup factor (S), computed as:

$$S = \frac{t_{\text{seq}}}{t_{\text{parallel}}} \quad (3)$$

and the parallel efficiency (E):

$$E = \frac{S}{p} \quad (4)$$

where t_{seq} is the mean execution time of the single-threaded CPU baseline, t_{parallel} is the mean execution time of the parallel version, and p represents the number of worker processes.

4.1 Experimental Setup

The experiments were conducted on a virtualized environment with the following hardware and software specifications:

- **Operating System:** Linux x86_64
- **CPU:** Virtualized 2-Core / 4-vCPU Host Processor
- **GPU:** NVIDIA Tesla T4 (Turing Architecture, 2,560 CUDA cores, 16 GB GDDR6 VRAM)
- **Software Stack:** Python 3.10, NumPy 1.24, Numba 0.58, CUDA Toolkit 12.x

In compliance with rigorous performance measurement guidelines:

- Two preliminary warm-up runs were executed and discarded to eliminate JIT compilation overhead and cold-cache effects.
- A minimum of five independent runs were recorded per configuration, reporting the mean, standard deviation, minimum, and maximum execution times.
- Disk file I/O and memory allocations were strictly excluded from kernel timings.
- Full functional validation was performed using `np.array_equal(seq_results, par_results)`.

4.2 CPU Multiprocessing Performance Evaluation

Table 1 presents the runtime and speedup obtained by the CPU Multiprocessing implementation across varying worker process counts ($p \in \{1, 2, 4\}$).

Workers (p)	Mean Time (s)	Std Dev (s)	Min Time (s)	Max Time (s)	Speedup	Efficiency
1	377.86	± 0.85	376.89	378.93	0.999x	99.9%
2	195.42	± 0.70	194.58	196.36	1.931x	96.6%
4	207.93	± 2.84	204.43	211.62	1.815x	45.4%

Table 1: CPU Multiprocessing execution times, speedup, and efficiency (Baseline $t_{\text{seq}} = 377.37$ s).

At $p = 2$, the implementation exhibits near-ideal linear scaling with an efficiency of 96.6%, confirming that POSIX shared memory effectively mitigates IPC bottlenecking. However, increasing p from 2 to 4 introduces a performance regression, with mean execution time rising from 195.42 s to 207.93 s ($S = 1.81\times$). This is directly attributable to the physical hardware constraints of the host machine: scheduling four active CPU-bound worker processes across two physical CPU cores induces severe context switching penalties and L3 cache thrashing.

4.3 CUDA GPU Performance Evaluation

The CUDA safety invariant kernel was evaluated on the NVIDIA Tesla T4 with a block size of 256 threads per block (TPB = 256). Six consecutive benchmark runs were collected, as detailed in Table 2.

Run	Execution Time (s)	Speedup vs Baseline	Correctness Validation
1	0.001484	254,281x	PASSED
2	0.001485	254,057x	PASSED
3	0.001492	252,868x	PASSED
4	0.001489	253,404x	PASSED
5	0.001518	248,602x	PASSED
6	0.001482	254,692x	PASSED
Mean ($\pm$ Std)	0.001492 s $\pm$ 0.000013 s	$\approx$ 253,000x	100% PASSED

Table 2: CUDA kernel execution time and speedup across 6 independent runs (TPB = 256).

The GPU implementation drops the verification time from over six minutes on CPU to just **1.49 milliseconds**, delivering an average speedup of $\sim$ **253,000 $\times$** . The high throughput is enabled by the compact 32-bit state representation, which ensures fully coalesced global memory access, minimal register pressure, and high Streaming Multiprocessor (SM) warp occupancy.

4.4 Global Comparison

Table 3 presents the global comparison between the sequential CPU baseline, the best CPU multiprocessing configuration, and the GPU CUDA implementation.

Architecture / Platform	Execution Units	Mean Runtime	Speedup	Validation
CPU Sequential (Baseline)	1 Core	377.37 s	1.00x	Reference
CPU Multiprocessing	1 Core	377.86 s	1.00x	PASSED
CPU Multiprocessing (Best)	2 Cores	195.42 s	1.93x	PASSED
CPU Multiprocessing (Oversubscribed)	4 Cores	207.93 s	1.81x	PASSED
GPU CUDA (Tesla T4)	256 TPB	0.00149 s (1.49 ms)	~ 253,000x	PASSED

Table 3: Global performance comparison across CPU and GPU architectures.

5 Conclusions

In this report, we presented the parallel acceleration of an exhaustive safety model checker for railway interlocking systems. We demonstrated that compacting states into 32-bit integers provides substantial memory efficiency and enables single-cycle bitwise invariant verification.

On multicore CPUs, using POSIX shared memory overcomes the Python GIL and delivers near-linear scaling (96.6% efficiency at 2 cores), while clearly delineating the performance penalty of CPU core oversubscription. On the GPU, the CUDA implementation drastically reduces computation time from 377.37 s to 1.49 ms ($\sim 253,000\times$ speedup), enabling real-time verification of complex interlocking state graphs that were previously intractable.

Future work will focus on porting the Breadth-First Search reachability state generator directly onto CUDA device kernels to eliminate host CPU generation bottlenecks, as well as investigating symbolic GPU-accelerated Binary Decision Diagrams (BDD).

References

- [1] E. M. Clarke, O. Grunberg, D. Kroening, D. Pezled, and K. Veith, *Model Checking*, 2nd ed., MIT Press, 2018.
- [2] K. M. Hansen, “Formal model checking of railway interlocking systems,” *Formal Aspects of Computing*, vol. 21, no. 1, pp. 123–141, 2009.
- [3] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pp. 1–6, 2015.
- [4] D. B. Kirk and W.-m. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, 3rd ed., Morgan Kaufmann, 2016.
- [5] NVIDIA Corporation, *CUDA C++ Programming Guide*, Version 12.x, 2023.